



## MODEL ARCHITECTURE GUIDE

# LLMs vs SLMs vs RLMs

Big, small, and thinking models: what each one actually is,  
and when to use which.



**LLM**

Large Model



**SLM**

Small Model



**RLM**

Reasoning Model



AI Coach John

Follow



 Repost

# What Is a Language Model?

One core idea, three design goals

A language model is a system trained to predict the next token in a sequence. That is genuinely it: you give it text, it produces a probability distribution over what comes next, picks a token, appends it, and repeats.

"The capital of France is" → " Paris" (0.94)

Everything else is a variation on this. The three classes differ by what they optimize for:



## LLM (Large Language Model)

Maximize general capability. Hundreds of billions of parameters, broad knowledge, strong at almost any task.



## SLM (Small Language Model)

Maximize efficiency. Roughly 0.5B to 14B parameters, designed to run cheaply on hardware you already own.



## RLM (Reasoning Language Model)

Maximize thinking quality. Trained to generate an extended chain of reasoning *before* answering.



AI Coach John

Follow



 Repost

# LLMs: What They Are

Large Language Models: the generalists

## Key Specifications

 **Scale:** Tens to hundreds of billions of parameters

 **Trained on:** Massive datasets spanning open internet, books, & code

 **Strength:** Broad general knowledge across almost any task without task-specific training

 **Examples:** Claude Opus, GPT-4, Gemini Ultra, Llama 70B+

## How it behaves

Ask it anything: summarize a legal document, write Python code, explain quantum mechanics, or draft an email: it performs well without any specialization.

## One Model, Multiple Domains

100B+ Params



Software



Legal



Math



Science



Finance



Creative

# LLMs: Tradeoffs

Power comes at a real cost

## ✓ Strengths

- ✓ Best-in-class quality on complex tasks
- ✓ Handles nuance & ambiguity
- ✓ Strong zero-shot performance
- ✓ Large context windows

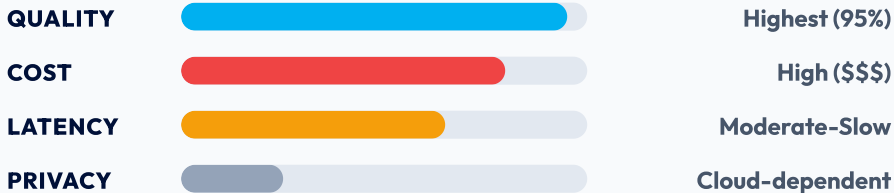
## ✗ Costs

- ✗ Expensive per request
- ✗ Higher latency
- ✗ Cloud-dependent privacy
- ✗ Overkill for simple tasks



**Tradeoff Summary:** Unmatched reasoning depth, but incurs heavy cloud compute cost & API rate limit risks.

## LLM CAPABILITY PROFILE



AI Coach John

Follow



 Repost

# LLMs: When to Use

Reach for an LLM when breadth matters more than cost

## ✓ Optimal Conditions Checklist

- ✓ Open-ended & varied tasks
- ✓ Broad knowledge needed
- ✓ Low-to-moderate volume
- ✓ Quality > per-token cost

## 🚀 Top Production Applications



Customer-facing chat assistant



RAG over diverse enterprise docs



Long-form content generation



Cross-stack code generation



Complex summarization with tone & nuance



Best choice when requirement volume is low to moderate and first-attempt accuracy is paramount.



AI Coach John

Follow



 Repost

# SLMs: What They Are

Small Language Models: the efficient specialists

## Key Specifications

-  **Scale:** Roughly 0.5B to 14B parameters
-  **Built via:** Distillation, quantization & efficient architectures
-  **Strength:** Fast, cheap, and runs locally on existing hardware
-  **Examples:** Phi-4 Mini (3.8B), Gemma 4, Qwen3, Llama 3.2

## The Important Nuance

"Small" spans a wide range. Not every SLM fits on a phone: target deployment depends heavily on parameter count and quantization tier.

0.5B - 2B (On-device)

4B - 8B (Laptop)

14B (Desktop GPU)

 Mobile / Edge

 Dev Laptop

 Workstation GPU



AI Coach John

Follow



 Repost

# SLMs: Which Sizes Run Where

"Runs on-device" depends entirely on the size tier

| Target Hardware                                                                                                              | Practical Size    | Reality Check                                   |
|------------------------------------------------------------------------------------------------------------------------------|-------------------|-------------------------------------------------|
|  <b>Phone (<math>\leq 4\text{GB}</math>)</b> | <b>Under ~2B</b>  | Smallest tier only (e.g. Gemma 4 e2b ~1.5GB Q4) |
|  <b>Laptop (8-16GB)</b>                      | <b>4B - 8B</b>    | Practical sweet spot for local development      |
|  <b>Desktop GPU (16-24GB)</b>                | <b>Up to ~14B</b> | Where Phi-4 class models reside                 |

## **Smartphone Memory Breakdown**

OS & Apps (2.5GB)

Available (~1.5GB)

Only sub-2B quantized models can fit inside the remaining 1.5GB memory window.



**Common Mistake:** A 7B model will NOT run on standard phones.  
Always verify the RAM footprint before designing on-device features.



**AI Coach John**

**Follow**



 **Repost**

# SLMs: Tradeoffs

Efficiency bought with capability

## ✓ Strengths

- ✓ Cheap / self-hosted
- ✓ Millisecond latency
- ✓ Full data privacy & offline
- ✓ Fast fine-tuning

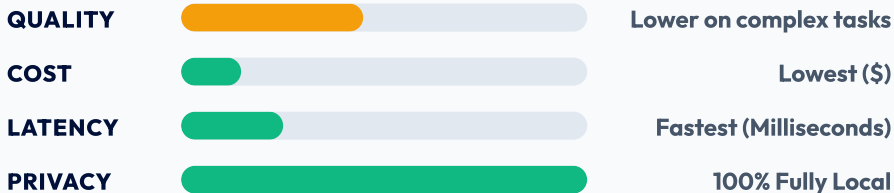
## ✗ Costs

- ✗ Weaker reasoning
- ✗ Smaller knowledge base
- ✗ Needs constrained decoding
- ✗ Shorter context windows



**Reliability Tip:** SLMs require structured JSON outputs & regex grammars to ensure zero output parsing errors.

## SLM CAPABILITY PROFILE



AI Coach John

Follow



 Repost

# SLMs: When to Use

Reach for an SLM when task is narrow and volume is high

## ✓ Optimal Conditions Checklist

- ✓ Narrow, defined task
- ✓ High request volume
- ✓ Privacy / offline required
- ✓ Sub-second latency

## ⚙️ Top Production Applications

👤 Query routing

👉 Intent classification

👤 PII detection / redaction

🔍 Structured field extraction

🏭 Edge & industrial inference

📱 On-device assistants



Self-hosting an SLM can reduce monthly API costs by over 90% at high request volumes.



AI Coach John

Follow



🔄 Repost

# RLMs: What They Are

Reasoning Language Models: the thinkers

## Key Specifications

-  **Scale:** Usually large: defining feature is *generation method*
-  **Trained via:** Reinforcement learning for extended reasoning chains
-  **Strength:** Substantially better on multi-step logic, math & code
-  **Examples:** OpenAI o-series, DeepSeek-R1, Claude extended thinking

### Standard LLM Flow:

Question → Immediate Answer Output

### RLM Thinking Flow:

Question → [Internal Reasoning: Hypothesize → Check Edge Cases → Self-Correct] → Final Answer

(Generates thousands of hidden internal thinking tokens before responding)



# RLMs: Tradeoffs

Accuracy bought with time and tokens

## ✓ Strengths

- ✓ Superior math & logic
- ✓ Self-corrects mid-reasoning
- ✓ High first-attempt accuracy
- ✓ Multi-step planning

## ✗ Costs

- ✗ Much slower latency
- ✗ High cost (thinking tokens)
- ✗ Over-thinks simple tasks
- ✗ Unpredictable delays



**The Over-Thinking Trap:** Asking an RLM simple questions wastes time and can introduce unnecessary complexity.

## RLM CAPABILITY PROFILE

QUALITY



Highest on hard tasks

COST



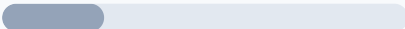
Highest (\$\$\$\$)

LATENCY



Slowest

PRIVACY



Cloud-dependent



AI Coach John

Follow



 Repost

# RLMs: When to Use

Reach for an RLM only when problem genuinely needs thinking

## ✓ Optimal Conditions Checklist

- ✓ Multi-step reasoning
- ✓ Accuracy >> cost/time
- ✓ Verifiable correct answer
- ✗ NOT for simple lookups

## 📺 Top Production Applications

🔧 Complex code debugging

√x Math & scientific proofing

🔗 Multi-constraint planning

🤖 Autonomous agent strategy

📈 Chained data analysis

🏗️ Code architecture decisions



Using an RLM for simple formatting or lookups costs 10x more for zero accuracy gain.



AI Coach John

Follow



↺ Repost

# Side-by-Side Comparison

All three classes, one comprehensive table

| Feature       | LLM           | SLM                | RLM                       |
|---------------|---------------|--------------------|---------------------------|
| Size          | 100B+ params  | 0.5B - 14B         | Usually large             |
| Optimized for | General power | Efficiency         | Reasoning depth           |
| Speed         | Moderate      | <b>Fastest</b>     | Slowest                   |
| Cost / Req    | High (\$\$)   | <b>Lowest (\$)</b> | <b>Highest (\$\$\$\$)</b> |
| Runs Local?   | Rarely        | <b>Yes</b>         | Rarely                    |
| Best at       | Broad tasks   | Narrow tasks       | Math & planning           |
| Typical Use   | Chat, RAG     | Routing, PII       | Debugging, Logic          |



Each class serves a distinct engineering objective. Selecting the right model optimizes both quality and unit economics.



AI Coach John

Follow



 Repost

# Choosing: A Decision Flow

Match the model class to the actual job

Is the task narrow, high-volume, or privacy-critical?

YES --> SLM ⚡

NO --> Check Next Question

Does it need genuine multi-step reasoning?

YES --> RLM ⚙️

NO --> LLM 🗣️



**Sanity Check:** Always ask "Could an SLM do this?" first. Defaulting to the largest model is the most common & expensive engineering mistake.



# The Real Pattern: Use All Three

Production systems route between model classes

## DYNAMIC ROUTER IMPLEMENTATION

```
def route_to_model(query, classifier):  
    task_type = classifier.predict(query)  
    if task_type in ("classify", "extract", "route"):  
        return slm # fast & ultra low cost  
    elif task_type in ("math", "complex_code"):  
        return rlm # deep reasoning engine  
    else:  
        return llm # general open-ended task
```

User Query



SLM Router



SLM (\$)

LLM (\$\$)

RLM (\$\$\$)



**Key Insight:** The router itself can be a small SLM, adding virtually zero overhead while maximizing system savings.



AI Coach John

Follow



 Repost

**⚠ MYTH BUSTING**

# Common Misconceptions

What people get wrong about these categories

**✗ Bigger is always better**

Fine-tuned SLMs often beat general LLMs on specific tasks.

**✗ All SLMs run on phones**

Only sub-2B quantized models realistically fit on mobile.

**✗ RLMs = LLMs + prompt**

RLMs are trained via RL specifically to reason internally.

**✗ SLMs are just worse LLMs**

Optimized for deployability & cost, not max breadth.

**✗ Use RLMs everywhere**

RLMs over-think simple questions, costing far more.

**✗ Categories are fixed**

Boundaries blur with toggleable thinking modes.



AI Coach John

Follow



 Repost

🎯 CORE TAKEAWAY

# Three tools, not three tiers

SLMs optimize for efficiency, LLMs for breadth, RLMs for reasoning depth. This is not a ranking from worst to best: it is three answers to three different questions.

⚡ **SLM:** Efficiency --> narrow, high-volume tasks

🧠 **LLM:** Breadth --> open-ended, varied requests

⚙️ **RLM:** Depth --> multi-step reasoning & math

**Save this reference guide**

Follow for more AI engineering breakdowns



AI Coach John

Follow



Repost